

Individual Report:

I was responsible for the Student App portion of the group work.

We used discord and WhenAvaliable to communicate and establish when everyone was free to meet. We used Google Meet most of the time to meet, and occasionally in person.

Pablo was team leader, created the GitHub and managed the meeting minutes. I fell ill causing my late to be a bit late but submitted it when we said we wanted our work to be done for. Other group members hadn't finished their work and didn't submit anything.

I attended the meetings when I could but, some were changed from online to in person which meant I couldn't make them.

Overall, I think we worked relatively well as a team answering questions when other team members were stuck and got together to complete our work.

We needed up having to use an extension for our work which I think enabled us to delay things.

Contribution to Group work:

I submitted my work when we set our deadline for us to combine and submit the following Friday. I however admit I should have shown the group my work earlier to supress any growing concerns that I hadn't done anything.

I made changes to work when Pablo and I were the only ones to have submit anything such as the file names to make them more user friendly.

Quality Requirements:

Security and Privacy Protection:

SEC – 1.1	Personal information must be protected	All user data fields must be protected by TLS 1.3 or higher for transmission between the mobile app and backend applications.
SEC – 1.2	Payment card details must not be stored within the USU system.	The application must integrate with payment providers using tokenisation, only non-sensitive tokens must be retained in the payment function.
SEC – 1.3	Access to event registration must be restricted to authorised services.	The EventParticipationService must enforce OAuth 2.0 validation for all registration API's.

2.2 – Performance

PER – 2.1	Time taken to process purchase after the user clicks 'Buy Ticket' must be fast.	The latency between submitting the purchase request and the payment gateway processing the transaction mustn't exceed a couple of seconds.
PER – 2.2	The backend processing of the payment call back must be immediate to confirm registration.	Must complete the task in less than a second.

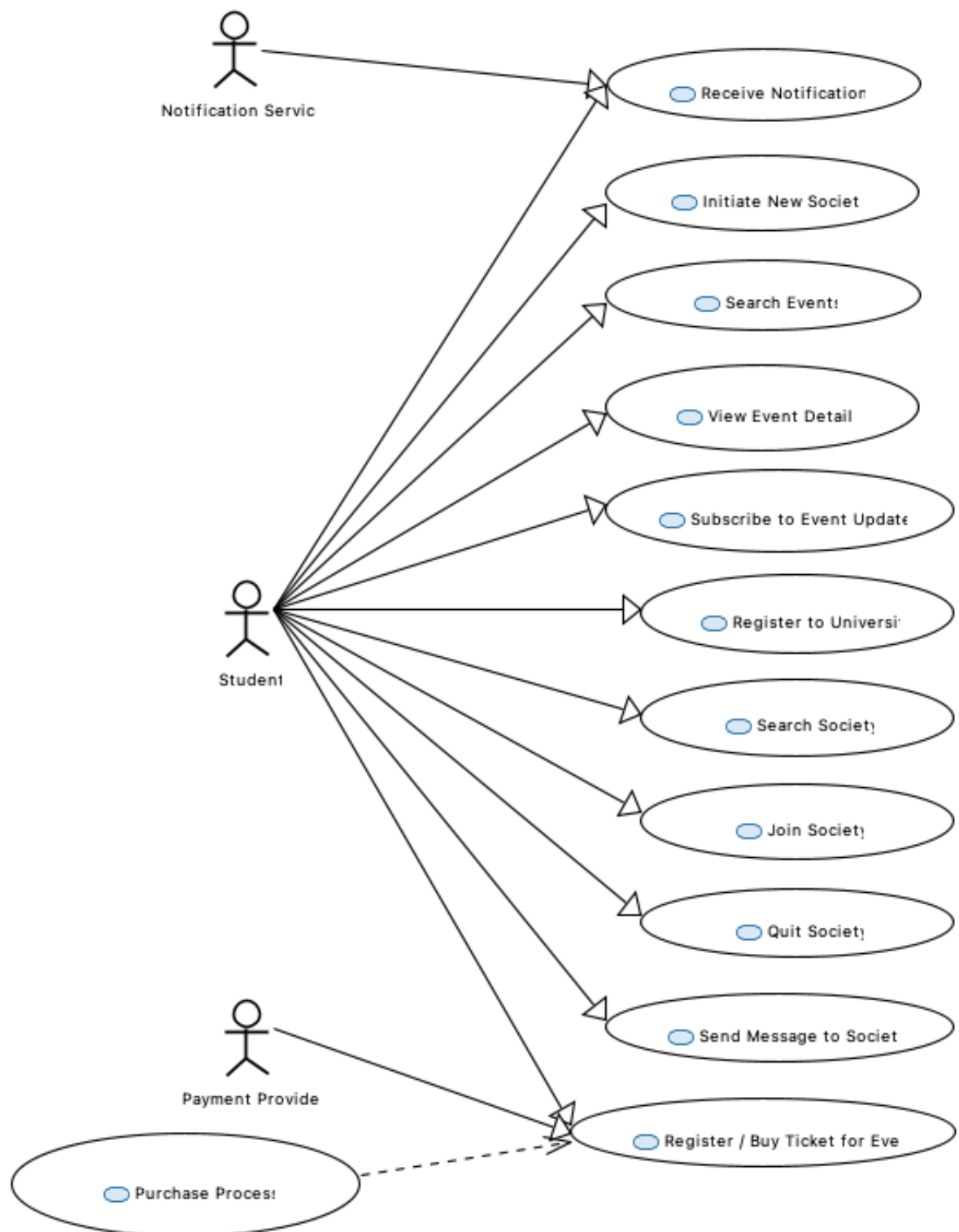
2.3 – Reliability

REL – 3.1	Buying tickets and registering to the platform must be highly available during peak times and when tickets are released.	Have retry mechanisms and verify transactions are succeeding.
REL - 3.2	Payment status changes must be resistant to external system failures.	Retry mechanisms and logic to verify the payment status from the external platform providers.

2.4 - Scalability

SCA – 4.2	Redis cache component must effectively reduce the database load.	Read requests for event availability must be served from the Redis cache in 95% of the cases during the first ten minutes of a sale.
-----------	--	--

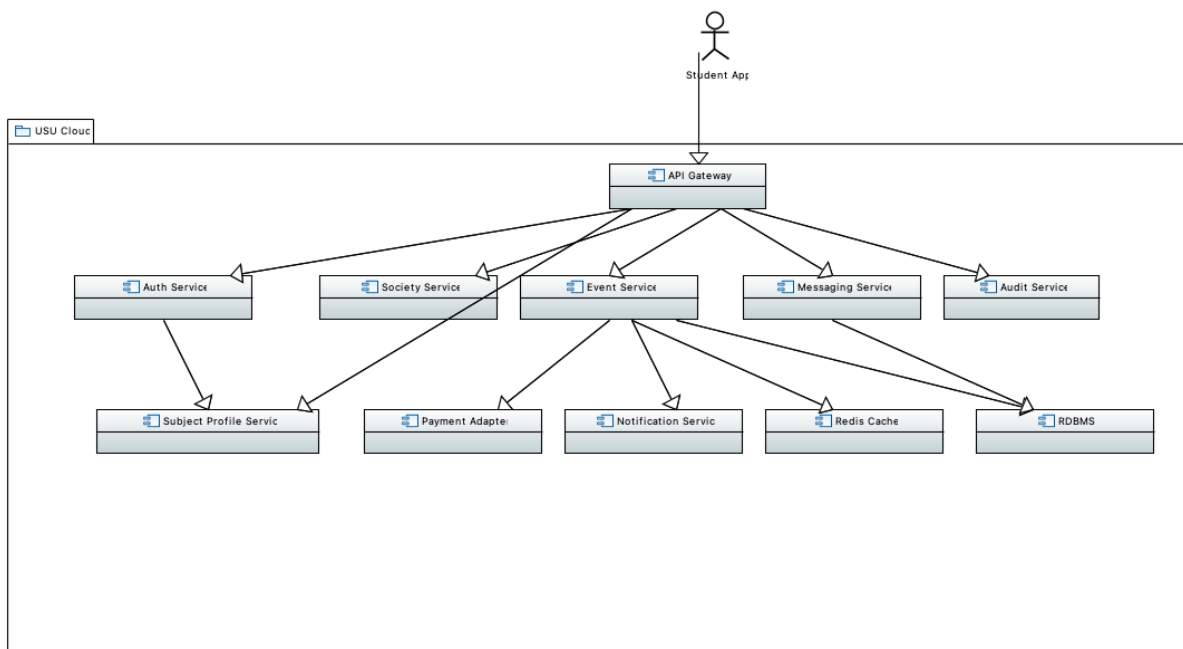
Use Case Model:



The use case diagram defines the functional requirements of the Student App, its dedicated to supporting the engagement with the user, with functions relating to event management, such as searching and viewing events, registering and buying tickets. It also is related to society management with tasks, relating to joining and quitting different societies.

It also has external integrations with systems such as payment gateways. The notification services is responsible for delivering updates and confirmations of bookings and new events available.

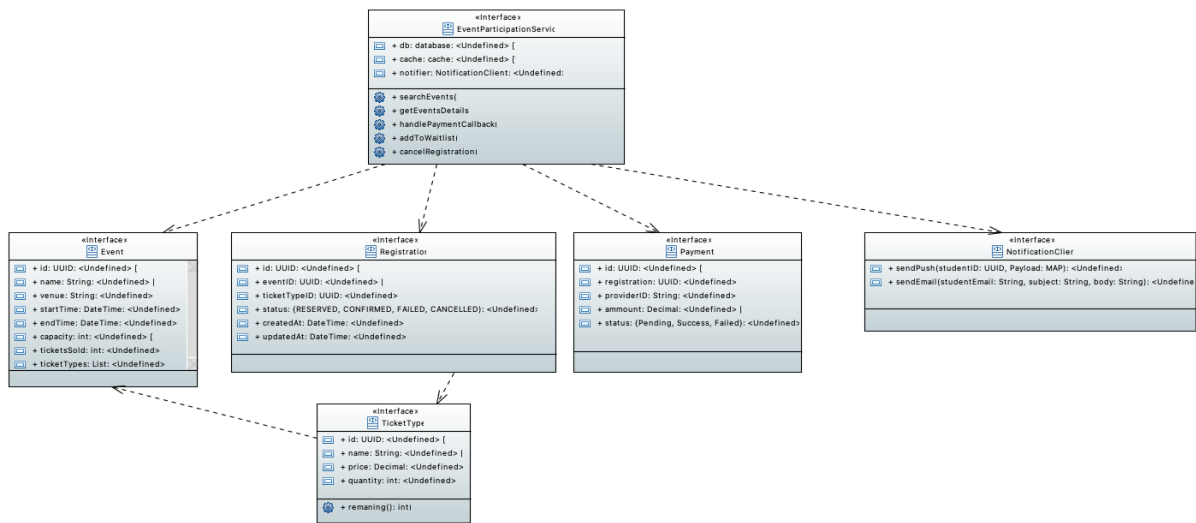
Activity Diagram: (Architectural Diagram)



This shows all the microservices within the USU cloud. All the client requests from the student app are first handled by the API gateway which then routes the requests to the destination such as the notification service or the auth service to log in etc.

This architecture promotes resilience and scalability by enabling independent scaling of microservices dependant on system load. For instance, if lots of people are logging in more auth services can be spun up, if lots of people are receiving notifications, more notification services can be spun up.

Component Diagram:



The UML component diagram shows the architectural design for the USU Student App. It focuses on the core microservices that the service needs. The diagram specifics the subsystems boundary by defining the public operations the service offers. This shows the dependencies that other services have on another service to successfully manage the event registration and ticketing process.